UNIVERSIDAD EAFIT

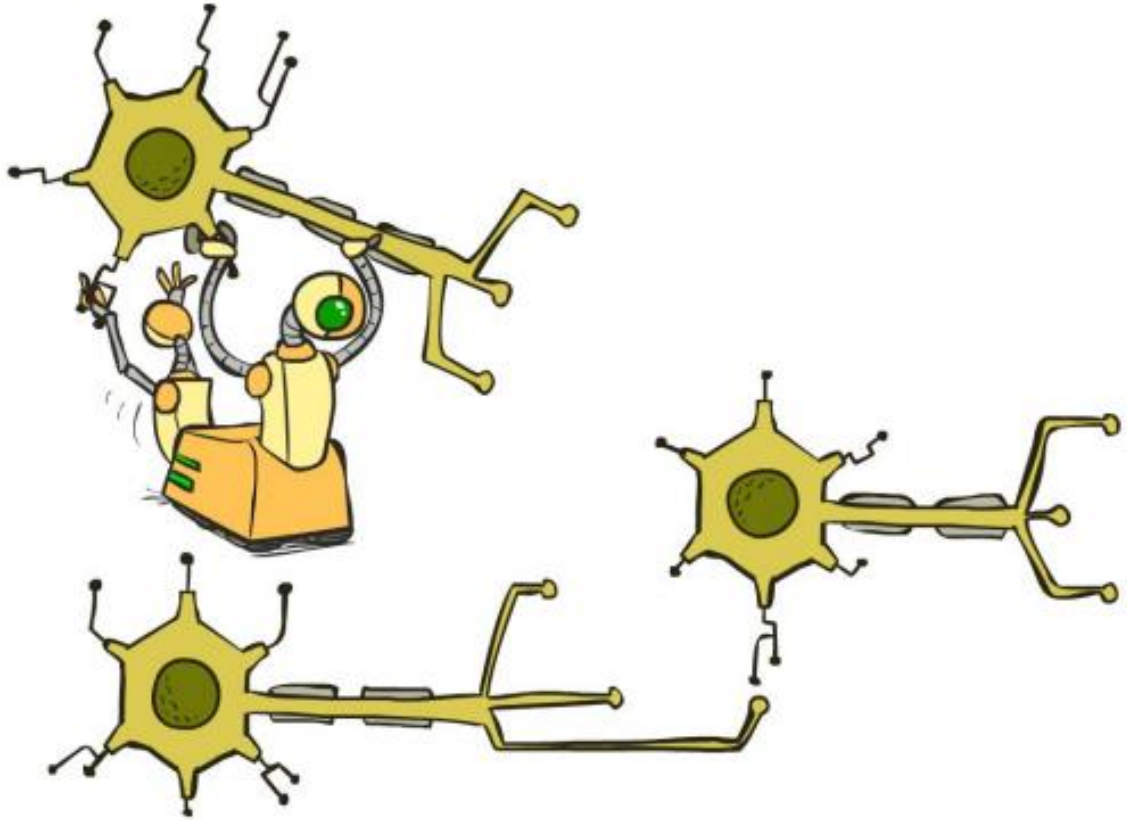
Escuela de Ciencias Aplicadas e Ingeniería

Artificial Intelligence

Lecture08 – Introduction to Deep Learning

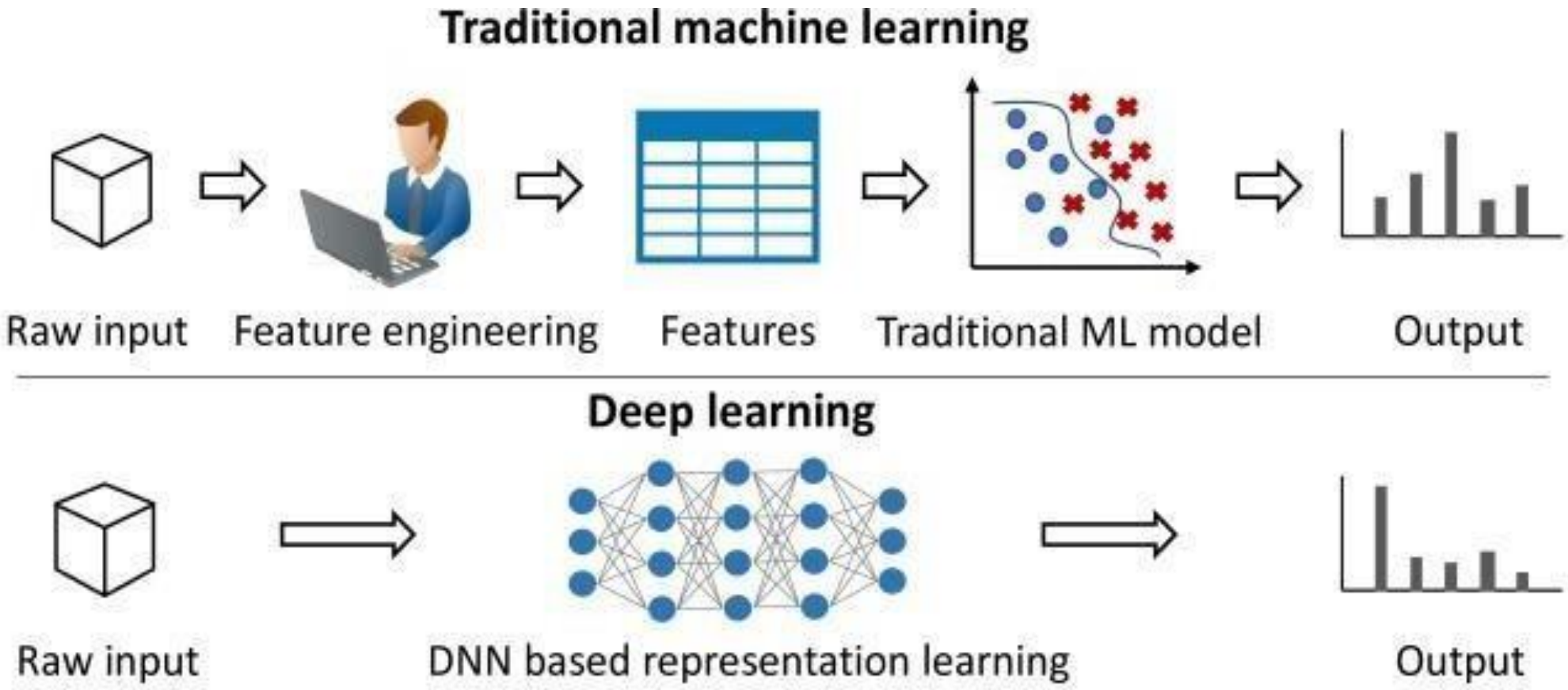
Content

- Introduction
- Deep Neural Networks (DNN)
- Convolutional Neural Networks (CNNs)



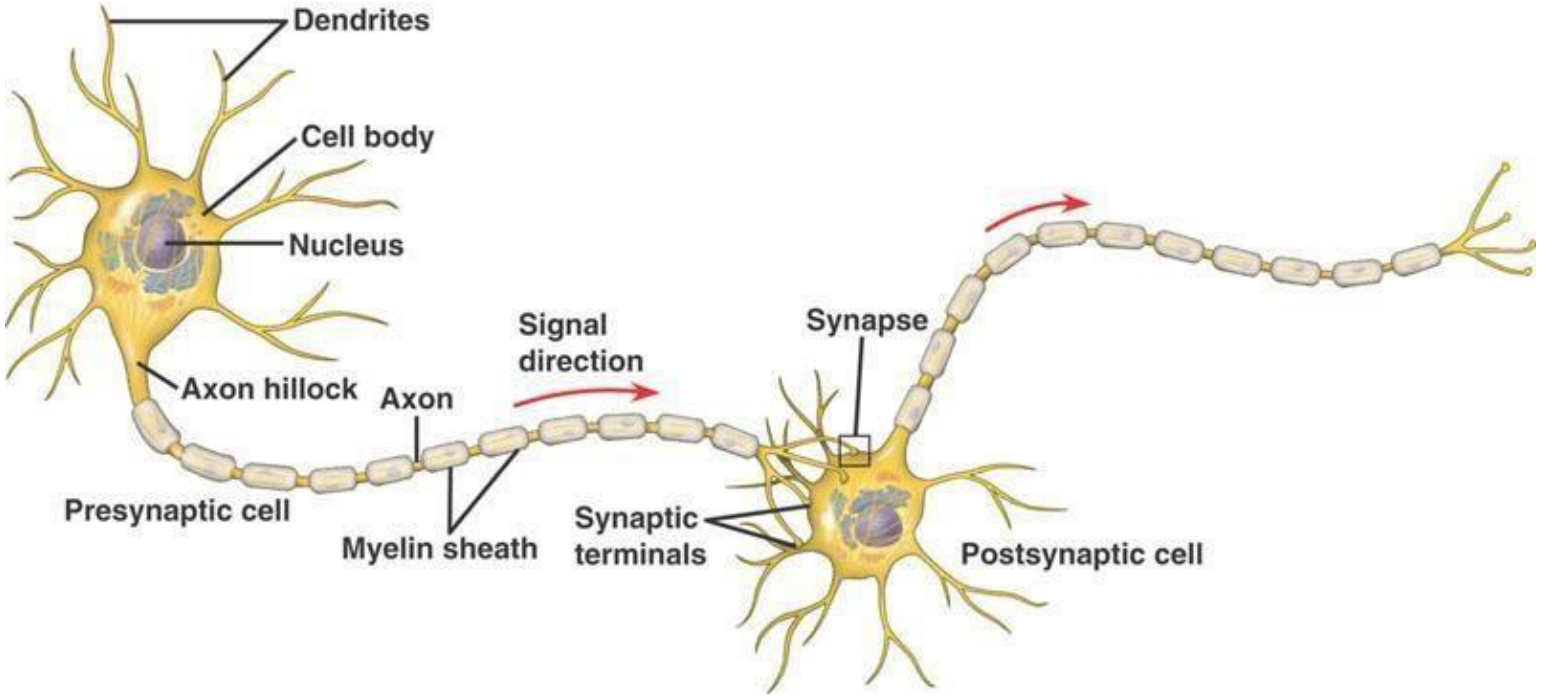
Introduction

How can we create features?

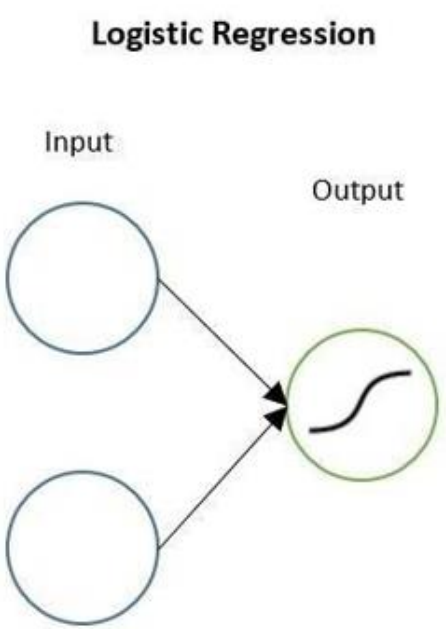


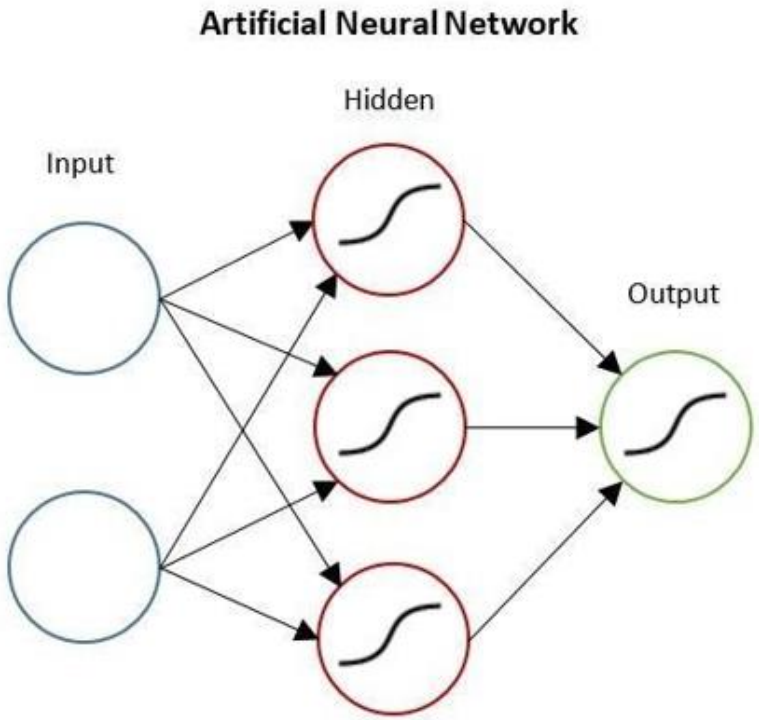
Introduction

Neural networks are powerful machine learning models inspired by the human brain. They learn complex patterns from data by adjusting millions of parameters across interconnected layers

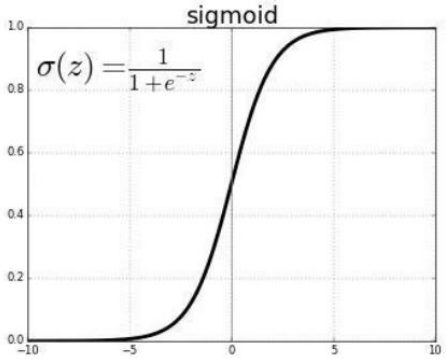


Introduction
Review



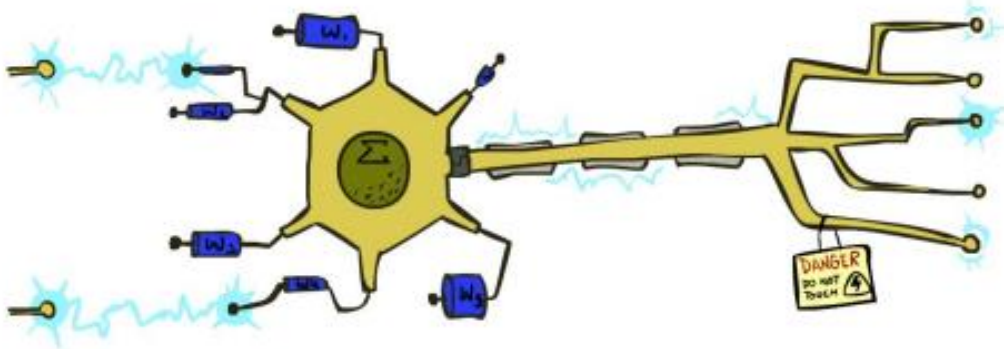


$$\hat{y} = h(\mathbf{x}) = \frac{1}{1 + \exp\left(-\mathbf{w}^T \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}\right)}$$



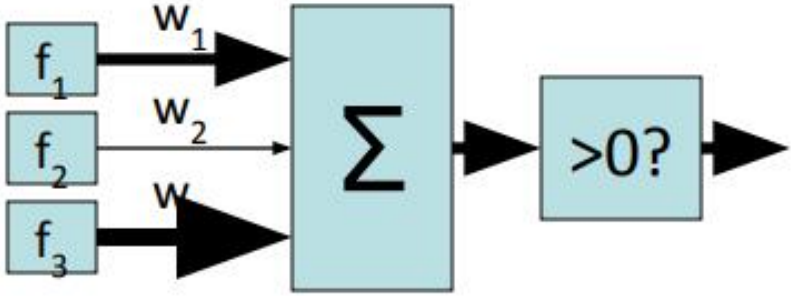
Introduction
Review

- Inputs are feature values
- Each feature has a weight
- Sum is the activation



$$\text{activation}_w(x) = \sum_i w_i \cdot f_i(x) = w \cdot f(x)$$

- If the activation is:
  - Positive, output +1
  - Negative, output -1

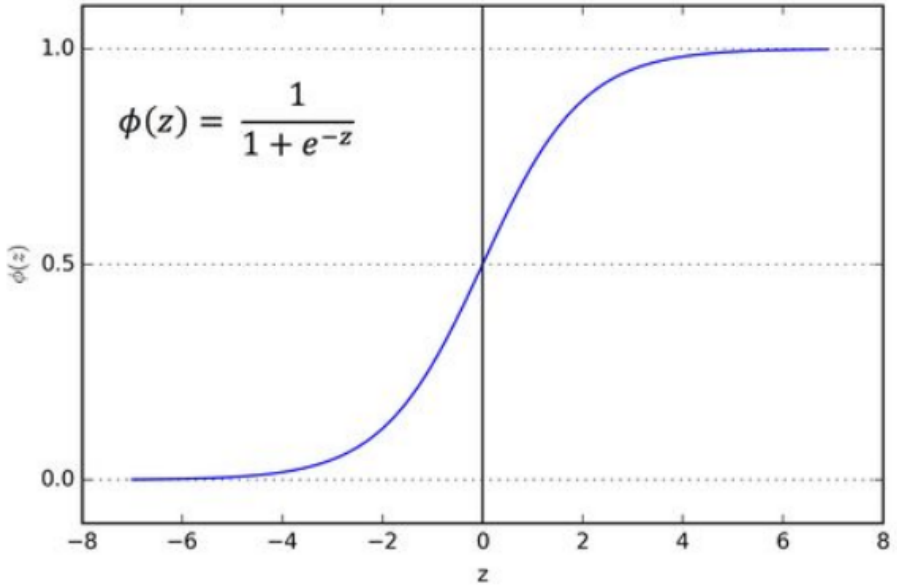


Introduction
Review

- Activation: $z = w \cdot f(x)$
- If z very positive □ want probability going to 1
- If z very negative □ want probability going to 0

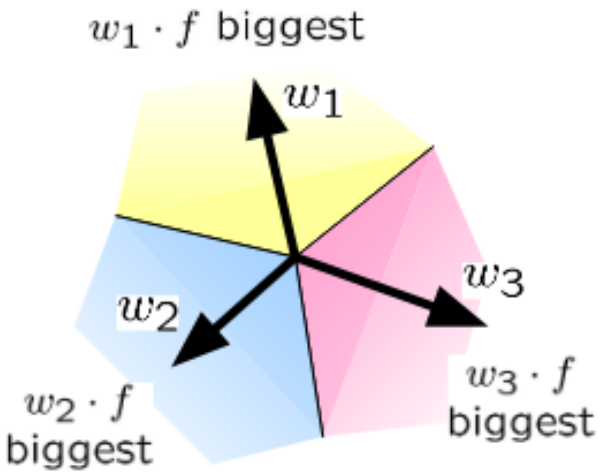
- Sigmoid function

$$\phi(z) = \frac{1}{1 + e^{-z}}$$



Introduction
Review

- Multi-class linear classification
  - A weight vector for each class: w_y
  - Score (activation) of a class y: $w_y \cdot f(x)$
  - Prediction w/highest score wins: $y = \arg\max_y \; w_y \cdot f(x)$

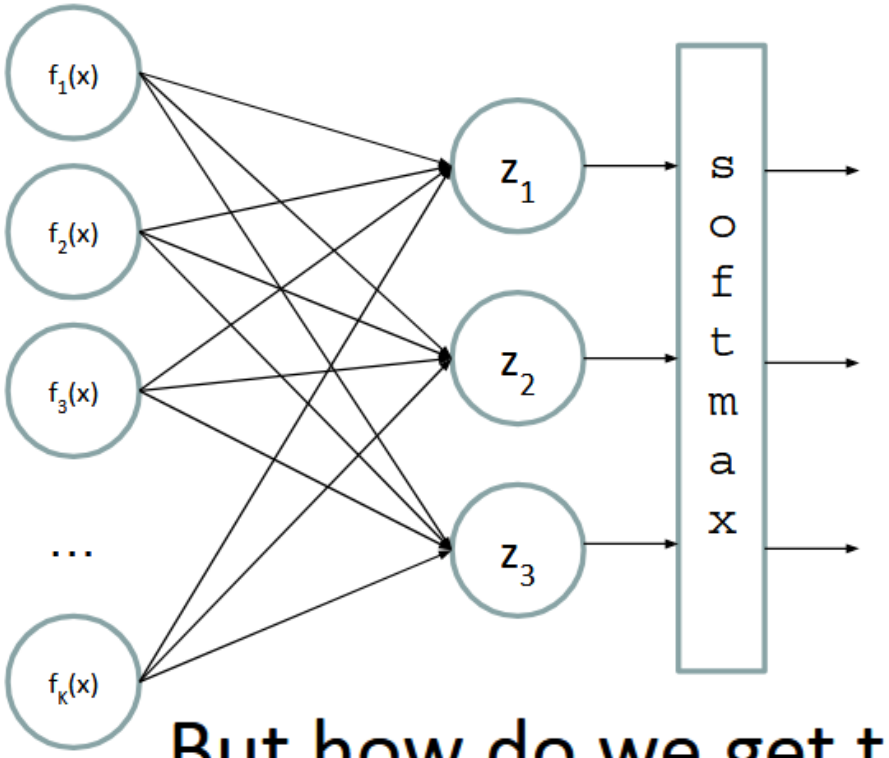


- How to make the scores into probabilities?

$$\underbrace{z_1, z_2, z_3}_{\text{original activations}} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1}+e^{z_2}+e^{z_3}}, \frac{e^{z_2}}{e^{z_1}+e^{z_2}+e^{z_3}}, \frac{e^{z_3}}{e^{z_1}+e^{z_2}+e^{z_3}}}_{\text{softmax activations}}$$

Introduction

$$h_\theta(x) = \begin{bmatrix} P(y=1|x;\theta) \\ P(y=2|x;\theta) \\ \vdots \\ P(y=K|x;\theta) \end{bmatrix} = \frac{1}{\sum_{j=1}^{K} \exp(\theta^{(j)\top} x)} \begin{bmatrix} \exp(\theta^{(1)\top} x) \\ \exp(\theta^{(2)\top} x) \\ \vdots \\ \exp(\theta^{(K)\top} x) \end{bmatrix}$$



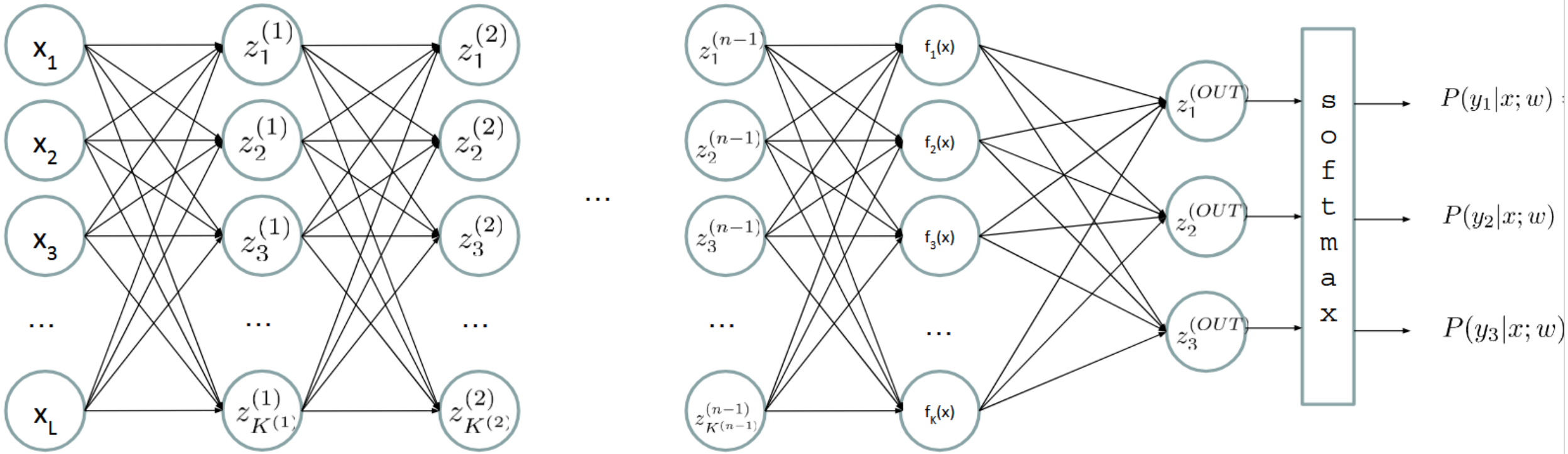
$$P(y_1|x;w) = \frac{e^{z_1}}{e^{z_1}+e^{z_2}+e^{z_3}}$$

$$P(y_2|x;w) = \frac{e^{z_2}}{e^{z_1}+e^{z_2}+e^{z_3}}$$

$$P(y_3|x;w) = \frac{e^{z_3}}{e^{z_1}+e^{z_2}+e^{z_3}}$$

But how do we get the features?

Deep Neural Network (DNN)



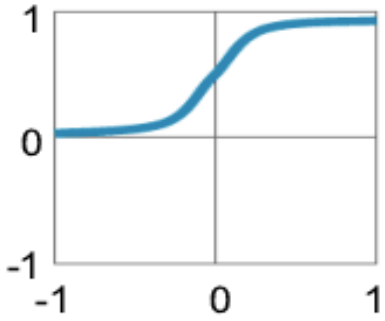
$$z_i^{(k)} = g\Big(\sum_j W_{i,j}^{(k-1,k)} z_j^{(k-1)}\Big)$$

g = nonlinear activation function (non-linearity is important! Otherwise you're just learning linear functions)

Deep Neural Network (DNN)

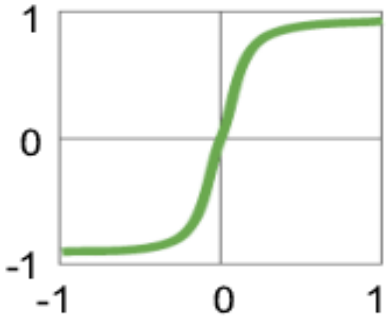
Traditional Non-Linear Activation Functions

Sigmoid



$y=1/(1+e^{-x})$

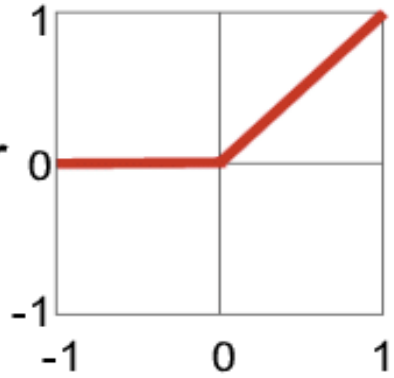
Hyperbolic Tangent



$y=(e^{x}-e^{-x})/(e^{x}+e^{-x})$

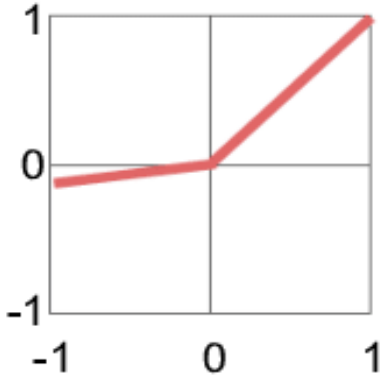
Modern Non-Linear Activation Functions

Rectified Linear Unit (ReLU)



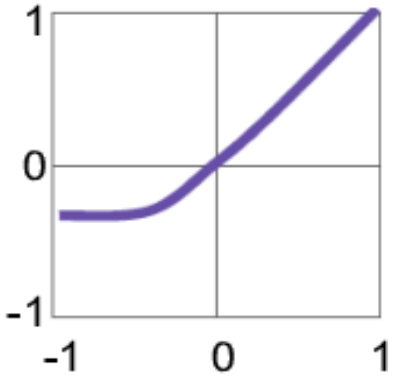
$y=\max(0,x)$

Leaky ReLU



$y=\max(\alpha x,x)$

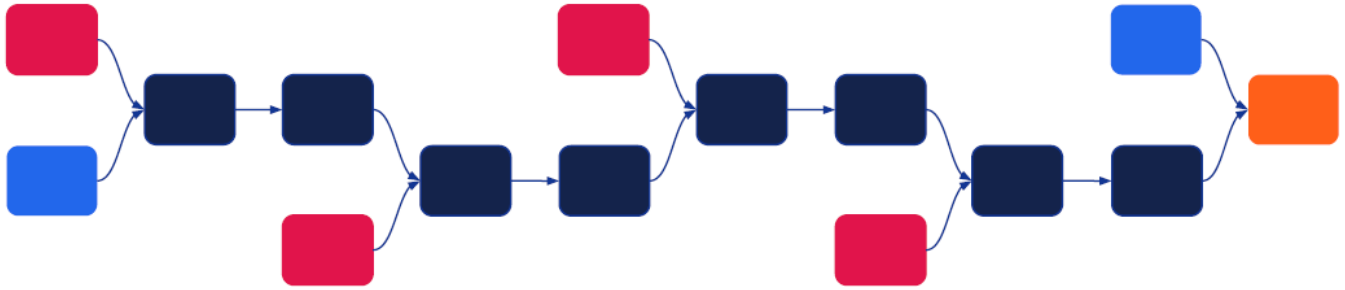
Exponential LU

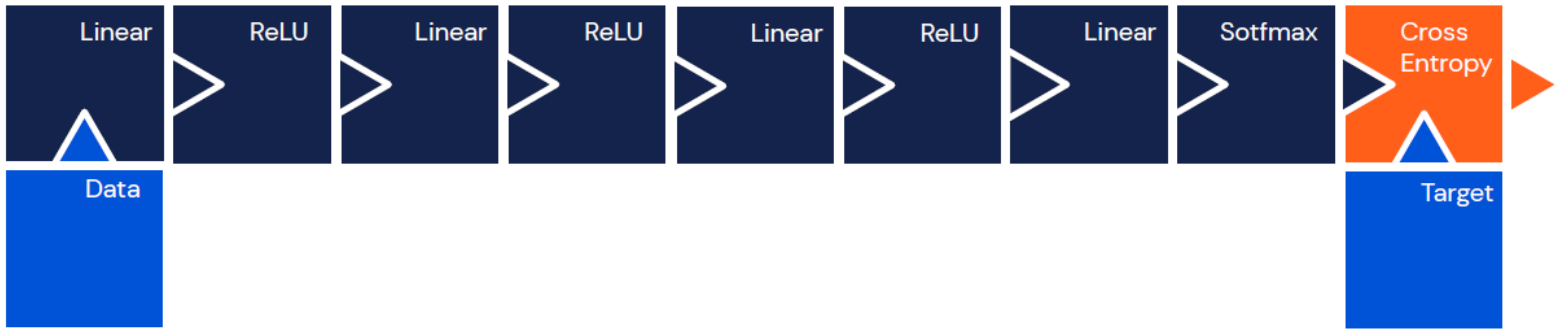


$$y=\begin{cases} x, & x\geq 0 \\ \alpha(e^{x}-1), & x<0 \end{cases}$$

α = small const. (e.g. 0.1)

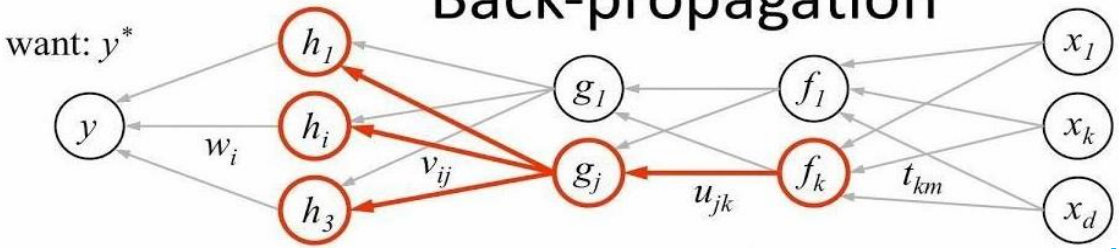
Deep Neural Network (DNN)



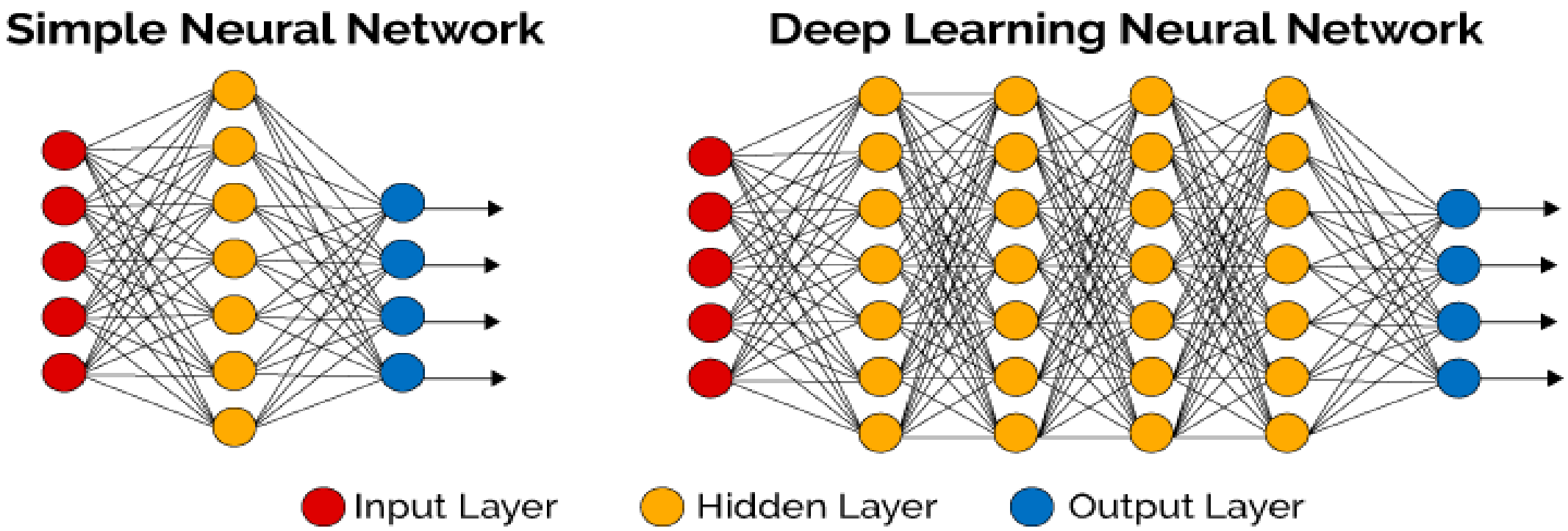


1. receive new observation $\boldsymbol{x} = [x_1 \ldots x_d]$ and target y^*
2. **feed forward:** for each unit g_j in each layer 1…L
   compute g_j based on units f_k from previous layer: $g_j = \sigma\left(u_{j0} + \sum_k u_{jk} f_k\right)$
3. get prediction y and error $(y\text{-}y^*)$
4. **back-propagate error:** for each unit g_j in each layer L…1

Back-propagation



Deep Neural Network (DNN)



Deep Neural Network (DNN)

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

- Theorem (Universal Function Approximators). A two-layer neural network with a sufficient number of neurons can approximate any continuous function to any desired accuracy.

- Practical considerations
  - Can be seen as learning the features
  - Large number of neurons
    - Danger for overfitting
    - (hence early stopping!)

Interactive exercise: http://playground.tensorflow.org

Deep Neural Network (DNN)

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

Class example and exercise

TensorFlow

PyTorch

Convolutional Neural Networks(CNNs)

Convolutional Neural Networks(CNNs)

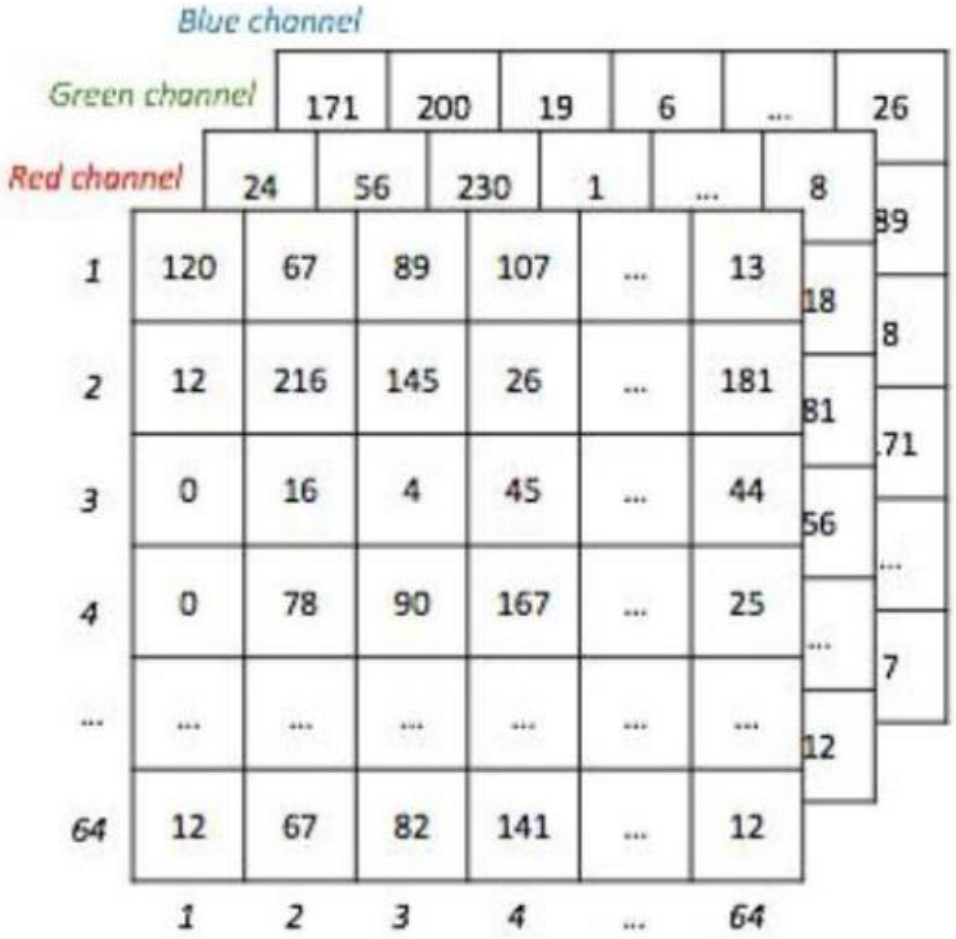


Image array: [64 x 64 x 3]

Convolutional Neural Networks(CNNs)

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

An image in gray scale only has 1 chanel…

Una imagen...

		0.6	0.6		
	0.6			0.6	
	0.6	0.6	0.6	0.6	
	0.6			0.6	

...es una matriz de pixeles.
El valor de los pixeles va de 0 a 255 pero se normaliza para la red neuronal de 0 a 1

Convolutional Neural Networks(CNNs)

UNIVERSIDAD EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

Image convolution

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

=

6		

7x1+4x1+3x1+
2x0+5x0+3x0+
3x-1+3x-1+2x-1
= 6

Imagen

Filtro o kernel

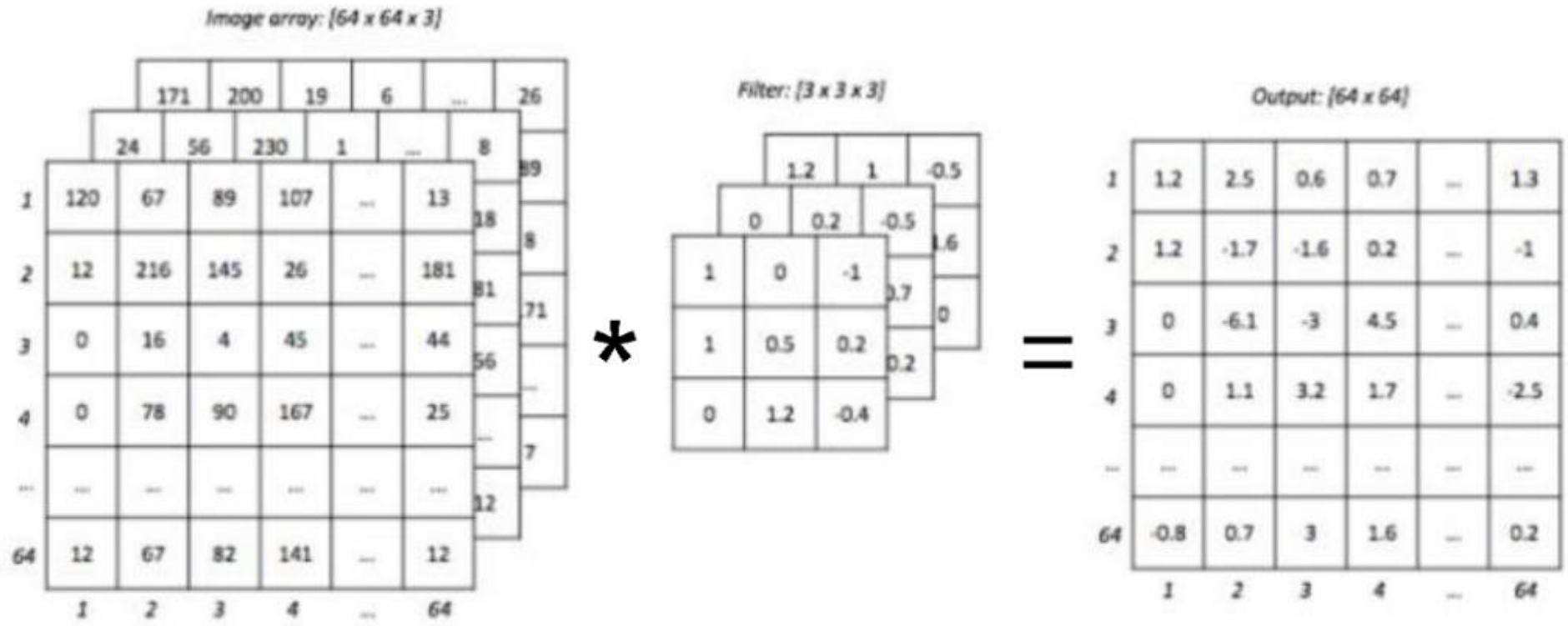
Resultado de la convolución

Convolutional Neural Networks(CNNs)

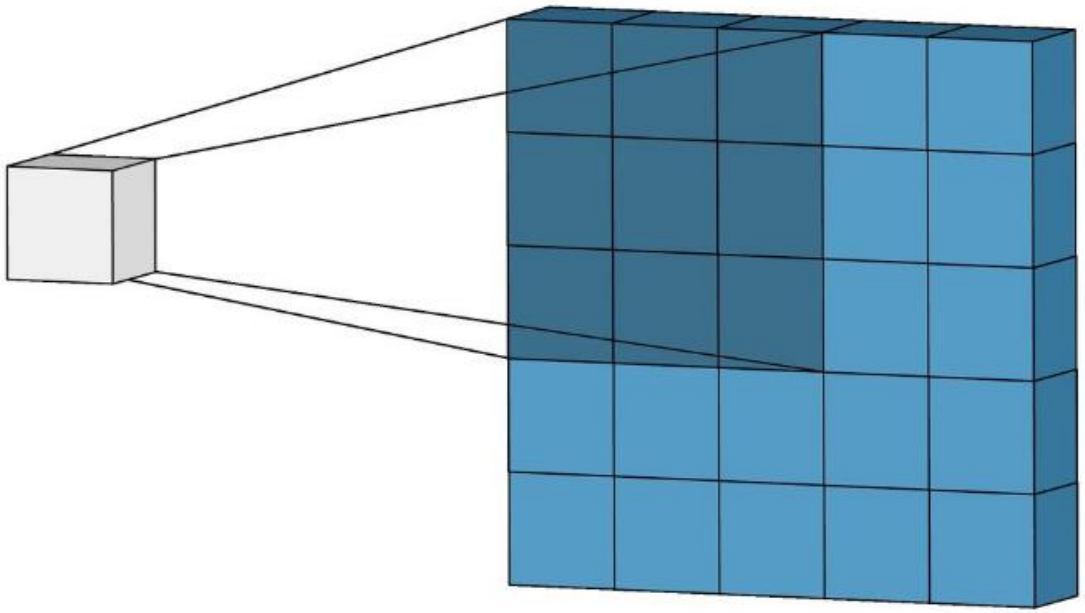
$$* \begin{pmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{pmatrix} =$$

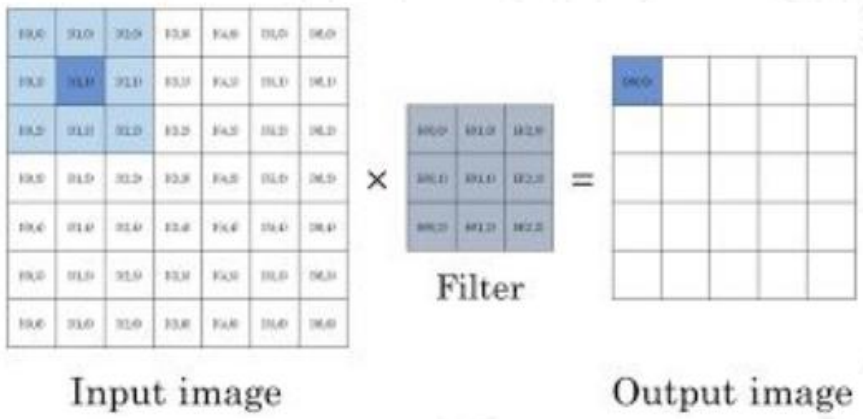
$$* \begin{pmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{pmatrix} =$$

Convolutional Neural Networks(CNNs)



Convolutional Neural Networks(CNNs)

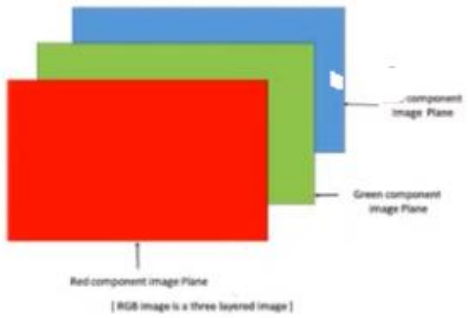


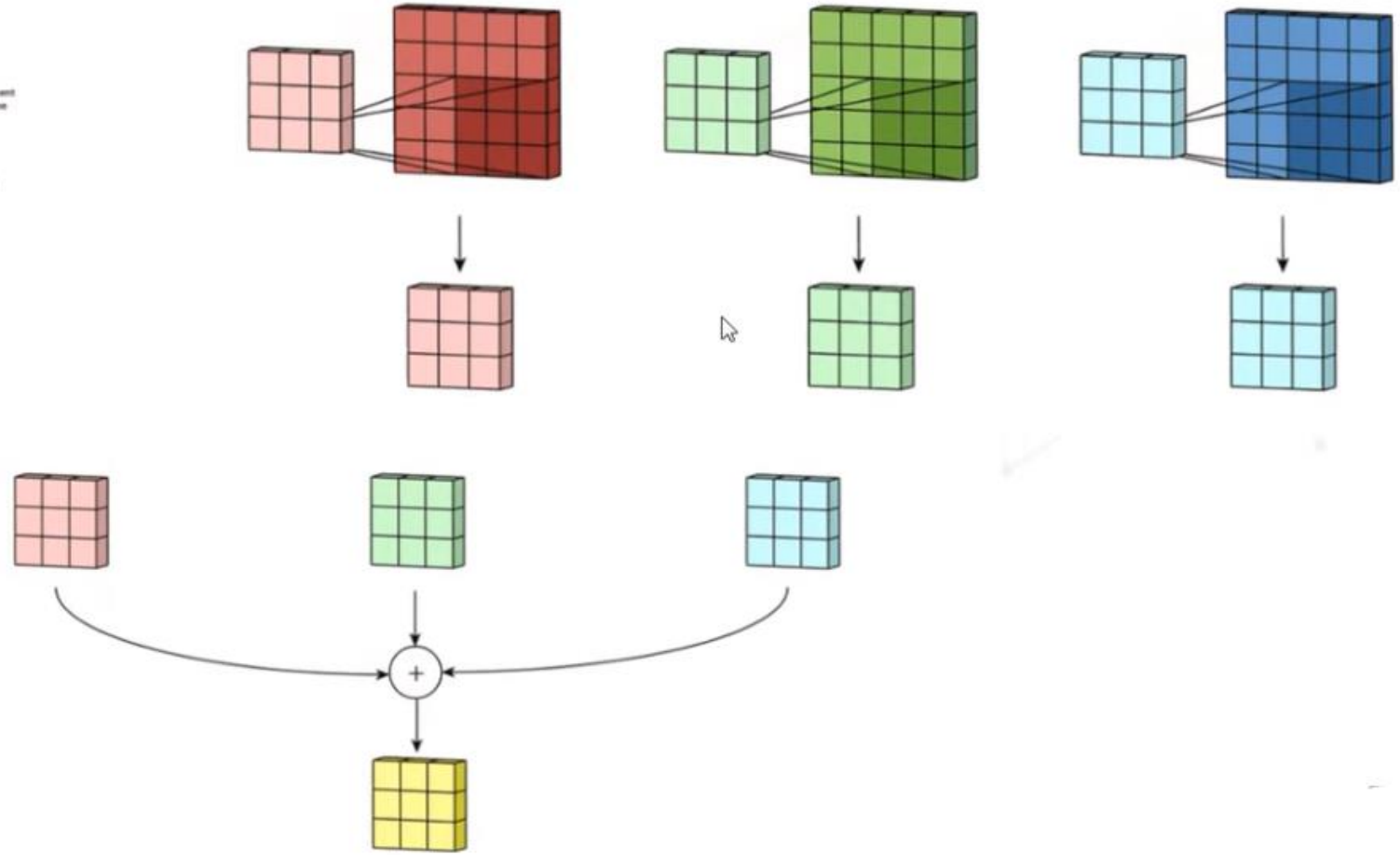


$$H_{out} = H_{in} - K + 1$$

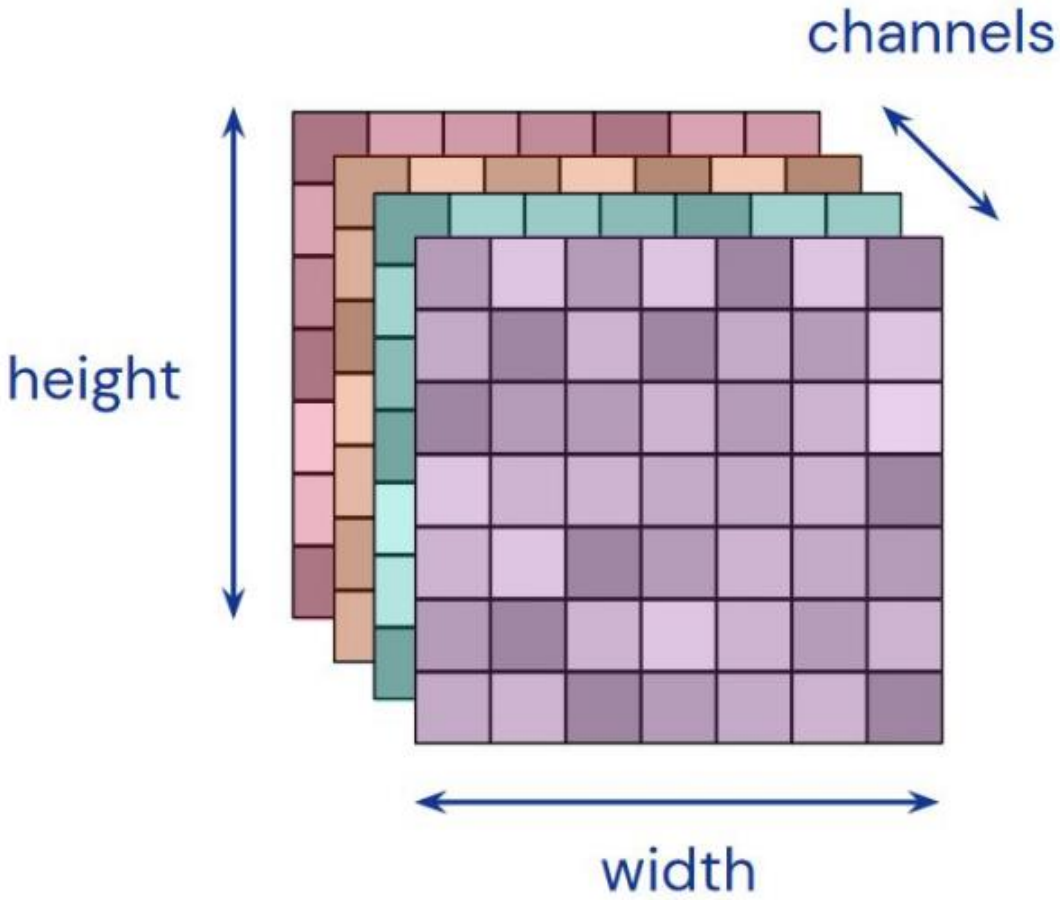
$$W_{out} = W_{in} - K + 1$$

Convolutional Neural Networks(CNNs)





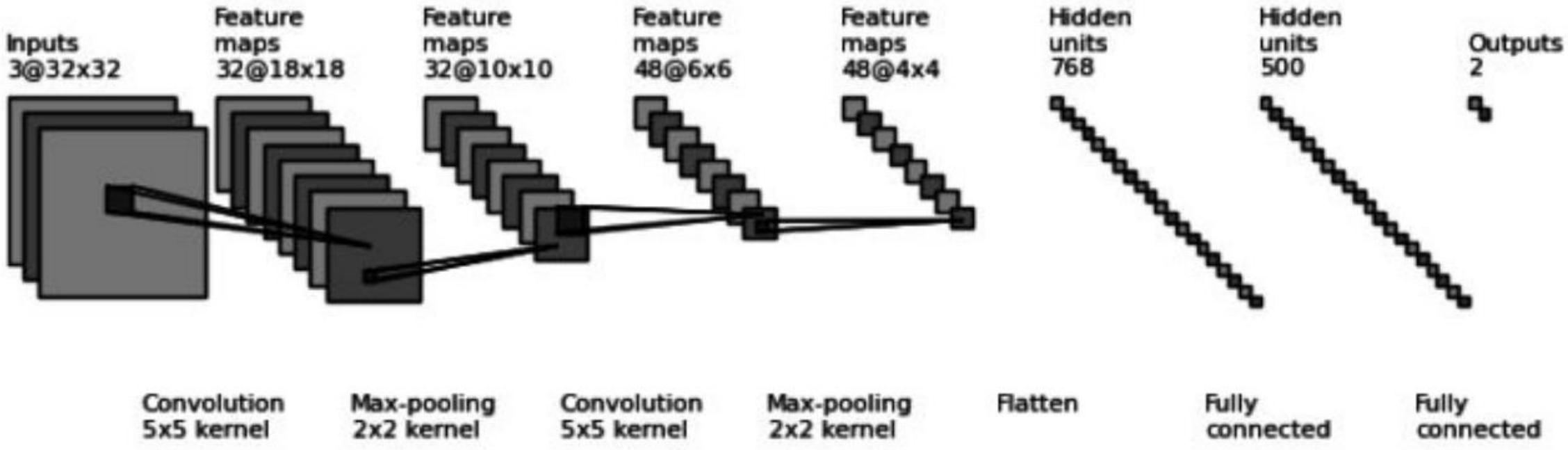
Convolutional Neural Networks(CNNs)



Convolutional Neural Networks(CNNs)

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

General Structure



Convolutional Neural Networks(CNNs)

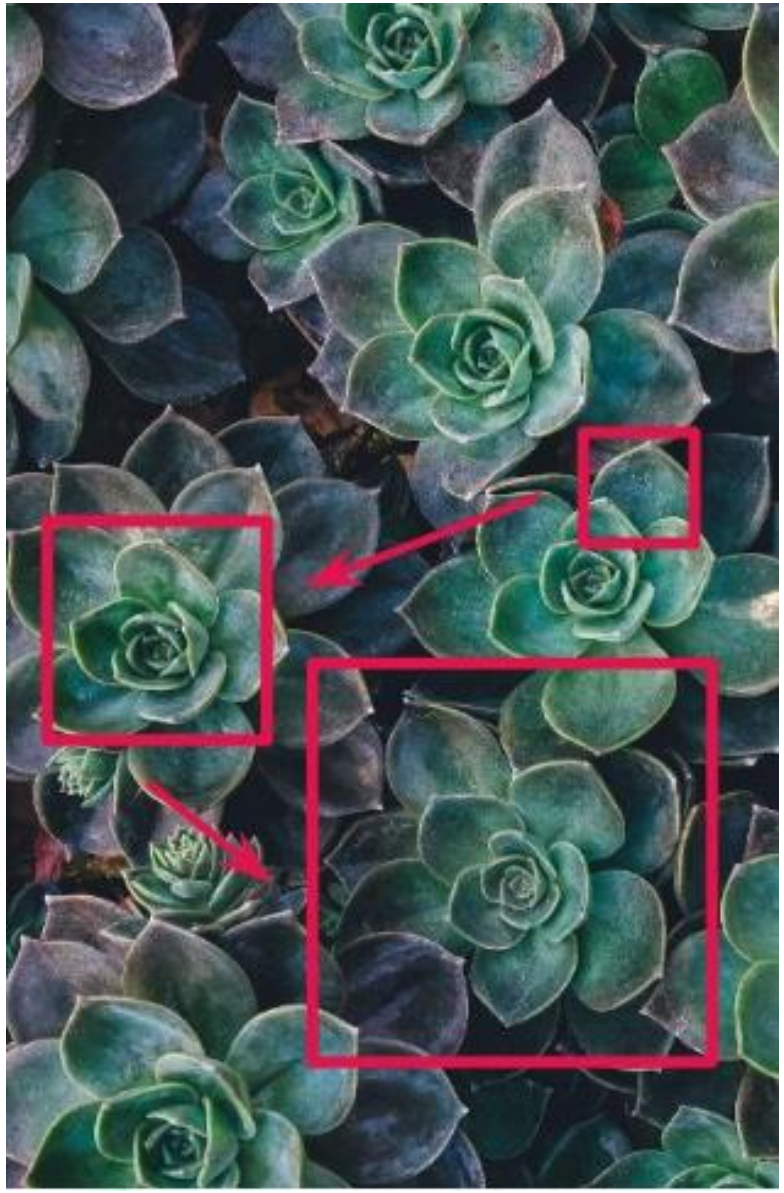
Is a bad idea to use DNN for image processing?

In general, using dense neural networks to process images is a bad idea for two main reasons:

- **Loss of pixel context.**
- **Excessive number of parameters** required to obtain a decent number of intermediate features.

A digital image is a 2D **grid of pixels.**

Convolutional Neural Networks(CNNs)



A dense neural network **does not receive information about whether two pixels are close to each other or far apart**, which leads to undesirable consequences:

- It is difficult to identify shapes formed by groups of nearby pixels.

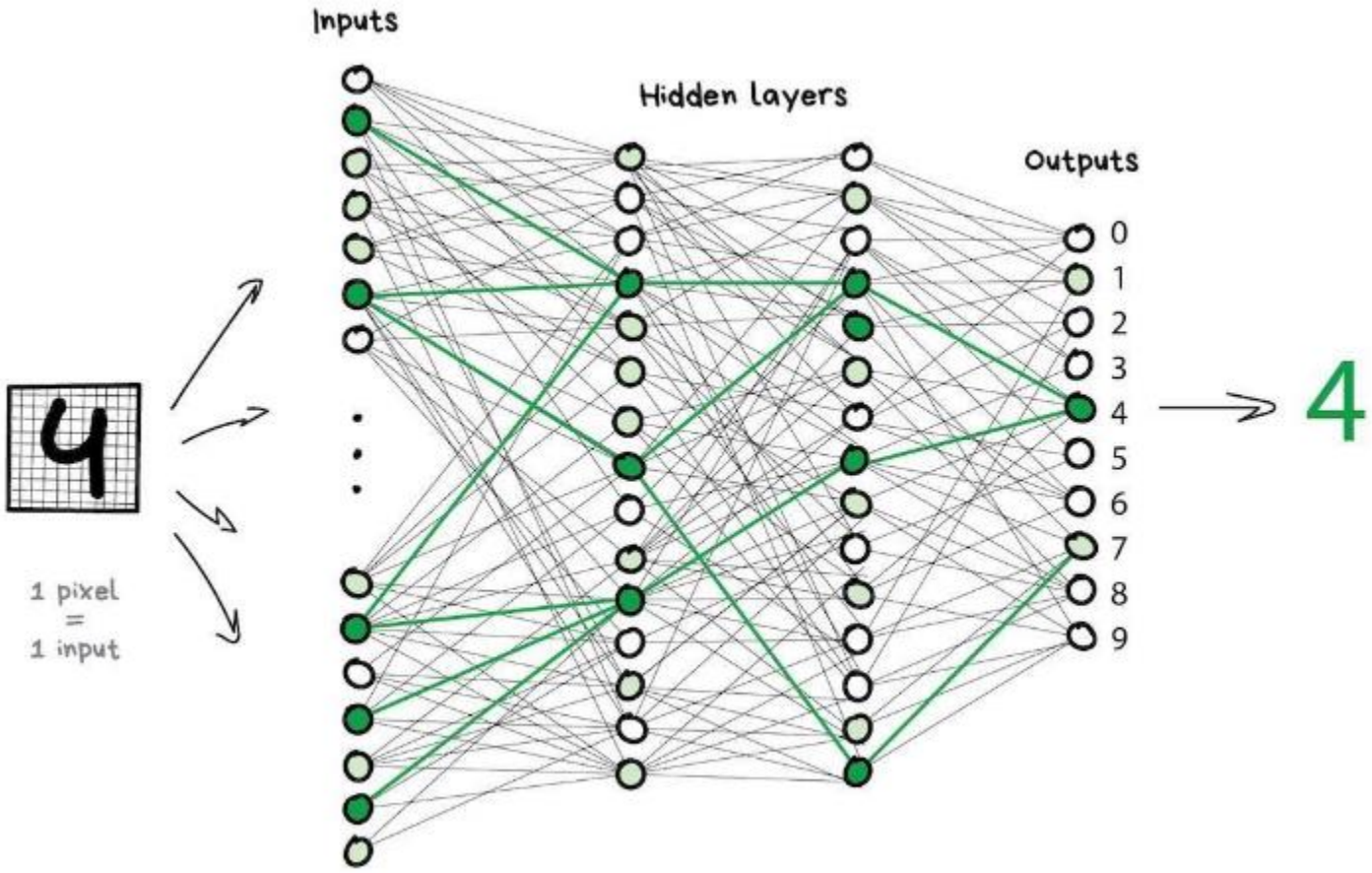
- The interaction between two pixels is considered even if they are very far apart.

- The way in which image features are processed depends on their position in the image.

Convolutional Neural Networks(CNNs)

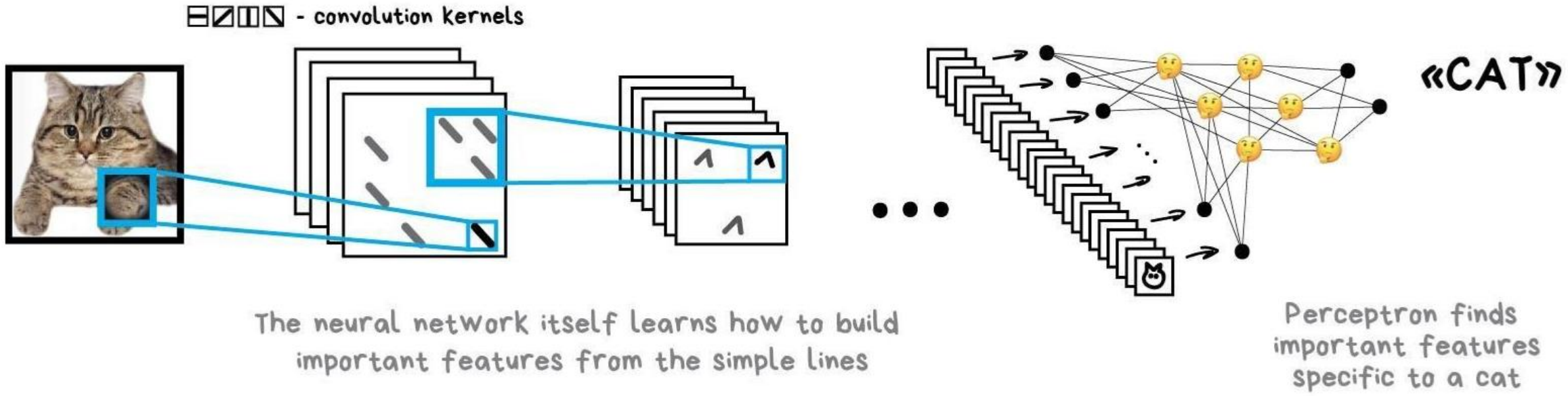
Suppose we want to apply a dense neural network to an image of **256 × 256 pixels.**

In order not to reduce the dimensionality of the features too much in the first hidden layer, suppose we use **2048 neurons** in that layer.



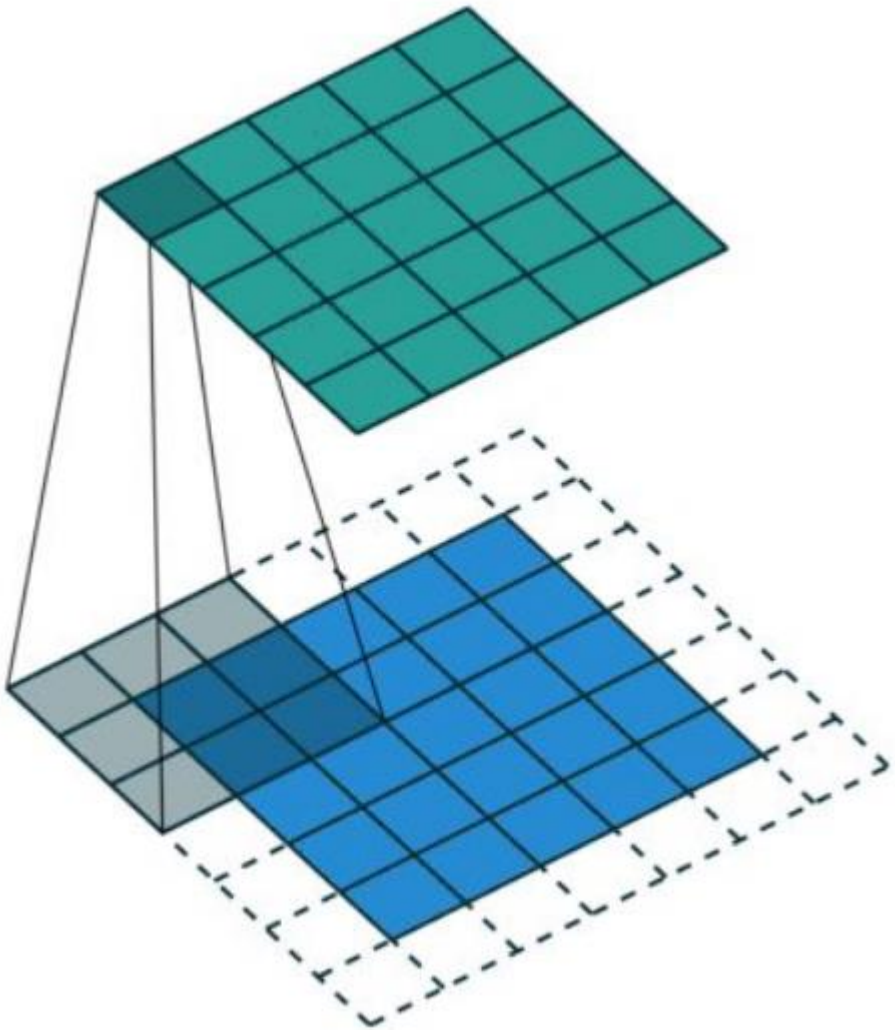
Around 134 millions of parameters for training!!

Convolutional Neural Networks(CNNs)

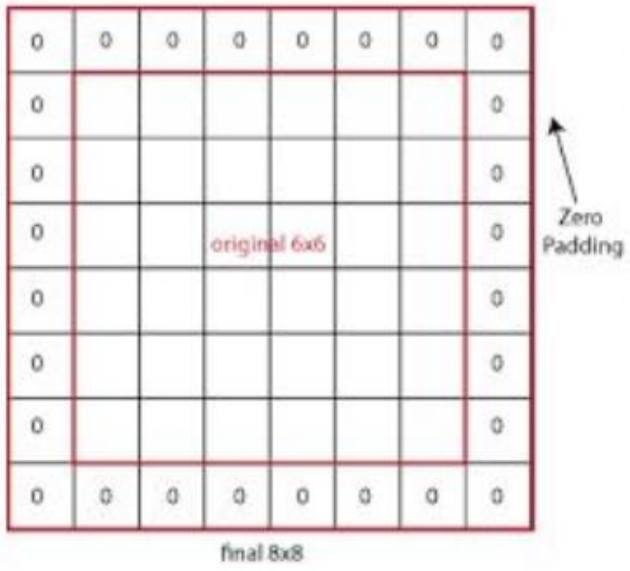


CONVOLUTIONAL NEURAL NETWORK (CNN)

Convolutional Neural Networks(CNNs)



Padding



$$W_{out} = W_{in} + 2p - K + 1$$

$$H_{out} = H_{in} + 2p - K + 1$$

Convolutional Neural Networks(CNNs)

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

Pooling

Entrada

7	3	5	2
8	7	1	6
4	9	3	9
0	8	4	5

pool máx. →

Salida

8	6
9	9

$$H_{out} = H_{in} - K + 1$$

$$W_{out} = W_{in} - K + 1$$

Convolutional Neural Networks(CNNs)

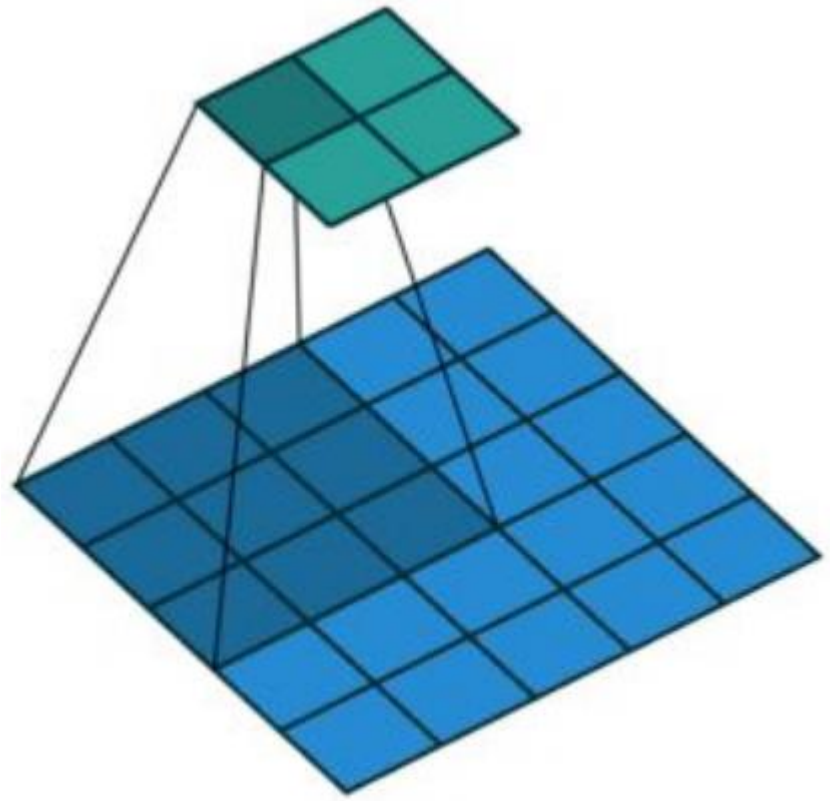
UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

Stride

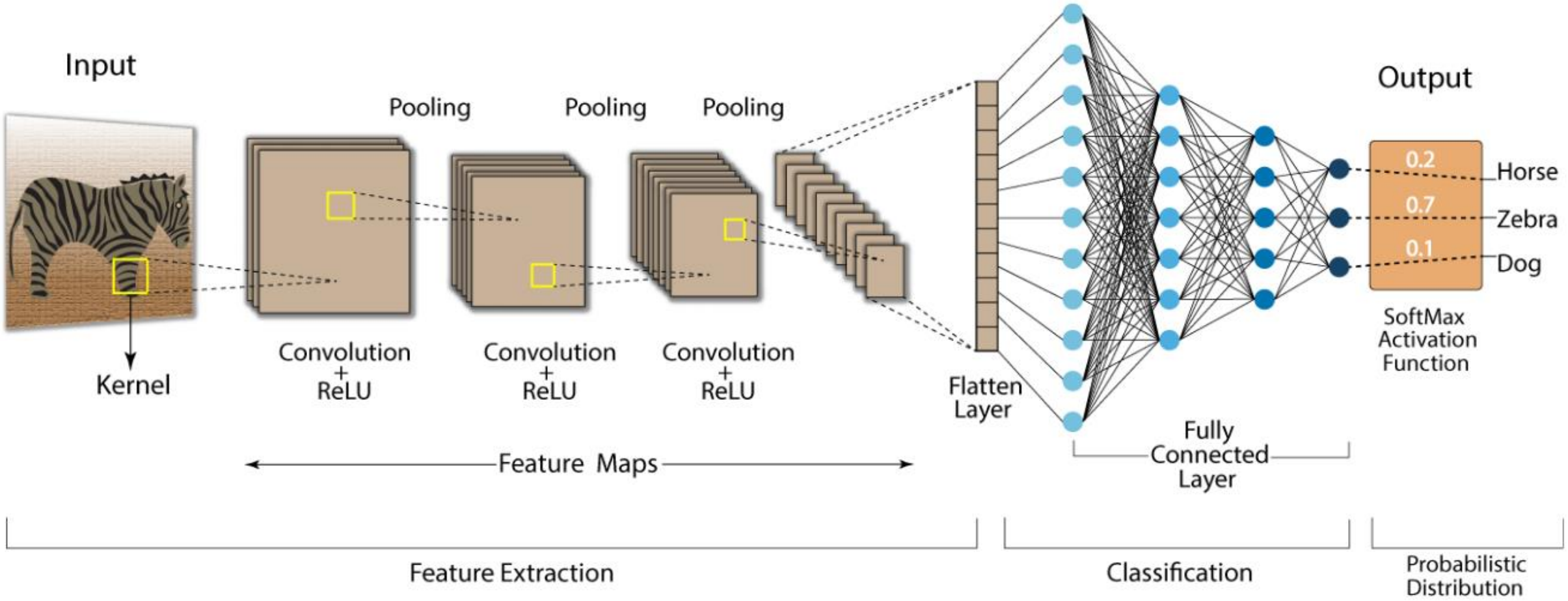
$$H_{out} = \frac{H_{in} + 2p - K}{s} + 1$$

$$W_{out} = \frac{W_{in} + 2p - K}{s} + 1$$

s: Stride
p: Padding
K: Kernel size
Hin: *Input Heigh* ***Hout:*** Output Heigh
Win: *Input Width* ***Wout:*** Output width



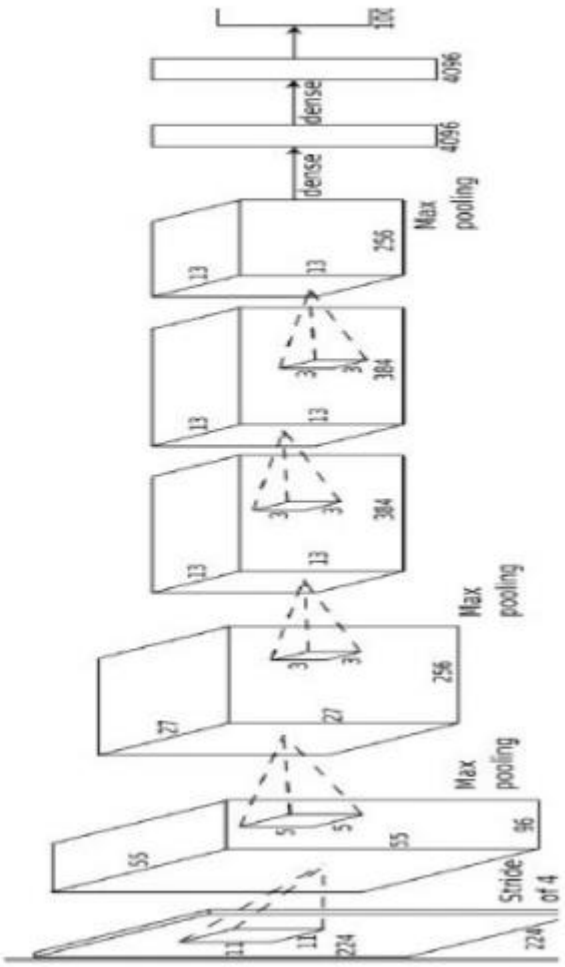
Convolutional Neural Networks(CNNs)



Convolutional Neural Networks(CNNs) AlexNet

- 1000 categories, 1.5 Million labeled training samples
- Won the 2012 ImageNet LSVRC. 60 Million parameters, 832M MAC ops

4M	FULL CONNECT	4Mflop
16M	FULL 4096/ReLU	16M
37M	FULL 4096/ReLU	37M
	MAX POOLING	
442K	CONV 3x3/ReLU 256fm	74M
1.3M	CONV 3x3ReLU 384fm	224M
884K	CONV 3x3/ReLU 384fm	149M
	MAX POOLING 2x2sub	
	LOCAL CONTRAST NORM	
307K	CONV 11x11/ReLU 256fm	223M
	MAX POOL 2x2sub	
	LOCAL CONTRAST NORM	
35K	CONV 11x11/ReLU 96fm	105M



Convolutional Neural Networks(CNNs)

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

Class example and exercise

TensorFlow

PyTorch